Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине Алгоритмы и структуры данных
Тема работы Динамическое программирование
Обучающийся Черненко Андрей Дмитриевич
Группа K4111c
Образовательная программа Разработка и эксплуатация промышленного бэкенда

Обучающийся _____ _____ Черненко А. Д.
(дата) (подпись) (Ф.И.О.)

Руководитель _____ _____ Мусаев А. А.
(дата) (подпись) (Ф.И.О.)

Санкт-Петербург, 2026

Содержание

ВВЕДЕНИЕ **2**

1 Задание 1. Эвакуация ценных вещей **2**

1.1 Условие 2

1.2 Исходные данные 2

1.3 Алгоритм решения 3

1.4 Результат 4

2 Задание 2. Оптимальное бинарное дерево поиска **4**

2.1 Условие 4

2.2 Исходные данные 4

2.3 Алгоритм решения 5

2.4 Результат 6

3 Задание 3. Раскрой бумажного бруска **6**

3.1 Условие 6

3.2 Исходные данные 6

3.3 Алгоритм решения 7

3.4 Результат 8

4 Сравнение результатов **8**

ЗАКЛЮЧЕНИЕ **9**

ВВЕДЕНИЕ

Целью данной лабораторной работы является изучение метода динамического программирования и его применения к задачам выбора набора объектов, построения оптимального бинарного дерева поиска и дискретного раскроя материала.

В работе решаются следующие задачи:

1. выбор предметов максимальной суммарной ценности при ограничениях по весу и времени;
2. построение бинарного дерева поиска с минимальной суммарной стоимостью обращений;
3. выбор заказов для раскроя бумажного бруска с учетом стоимости каждого реза.

Все алгоритмы реализованы на языке C#.

1 Задание 1. Эвакуация ценных вещей

1.1 Условие

В здании начался пожар, и имеется ровно T минут на эвакуацию ценных вещей. Для каждого предмета i заданы вес w_i, ценность v_i и время выноса t_i. Рюкзак выдерживает максимальный вес W. Требуется выбрать набор предметов максимальной суммарной ценности, не превысив одновременно ограничения W и T.

1.2 Исходные данные

Для проверки алгоритма использованы следующие предметы:

Таблица 1: Параметры предметов

№	Вес, кг	Ценность, $	Время, мин
1	3	6	4
2	4	7	5
3	5	10	6
4	2	4	3
5	6	12	7

Ограничения: $W = 10$ кг, $T = 12$ мин.

1.3 Алгоритм решения

Состояние динамического программирования задаётся величиной $d[i][w][t]$, равной максимальной ценности, которую можно получить, рассмотрев первые i предметов при доступных ограничениях по весу w и времени t.

Для каждого предмета рассматриваются два варианта:

1. не брать предмет и сохранить предыдущее значение;
2. взять предмет, если хватает веса и времени, и добавить его ценность.

Переход имеет вид:

$$d[i][w][t] = \max\left(d[i-1][w][t], d[i-1][w-w_i][t-t_i] + v_i\right).$$

```
static int Evacuation((int w, int v, int t)[] items,
                      int maxWeight, int maxTime)
{
    int n = items.Length;
    int[,,] dp = new int[n + 1, maxWeight + 1, maxTime + 1];

    for (int i = 1; i <= n; i++)
    {
        var item = items[i - 1];

        for (int w = 0; w <= maxWeight; w++)
        for (int t = 0; t <= maxTime; t++)
        {
            dp[i, w, t] = dp[i - 1, w, t];

            if (item.w <= w && item.t <= t)
```

```
            {
                int value = dp[i - 1, w - item.w, t - item.t]
                            + item.v;
                dp[i, w, t] = Math.Max(dp[i, w, t], value);
            }
        }
    }

    return dp[n, maxWeight, maxTime];
}
```

Листинг 1: Динамическое программирование для эвакуации

1.4 Результат

Оптимальный набор состоит из предметов 2 и 5. Его характеристики:

$$w = 4 + 6 = 10 \text{ кг}, \qquad t = 5 + 7 = 12 \text{ мин}, \qquad v = 7 + 12 = 19 \text{ \$}.$$

Таким образом, максимальная суммарная ценность равна 19 долларов.

Временная сложность алгоритма составляет $O(nWT)$, пространственная сложность --- $O(nWT)$.

2 Задание 2. Оптимальное бинарное дерево поиска

2.1 Условие

Даны отсортированные ключи и частоты обращений к ним. Время доступа к ключу равно глубине ключа в дереве плюс один. Требуется построить бинарное дерево поиска, минимизирующее суммарную стоимость:

$$C = \sum_i (\text{depth}(i) + 1) \cdot p_i.$$

2.2 Исходные данные

Использованы ключи 10, 20, 30, 40 с частотами обращений 3, 2, 4, 1 соответственно.

Таблица 2: Ключи и частоты обращений

Ключ	10	20	30	40
Частота	3	2	4	1

2.3 Алгоритм решения

Для каждого отрезка ключей $[i, j]$ перебирается возможный корень r. При выборе корня глубина всех ключей текущего отрезка увеличивается на единицу, поэтому к стоимости поддеревьев добавляется сумма частот отрезка:

$$cost[i, j] = \min_{r=i}^{j} \left(cost[i, r-1] + cost[r+1, j] + \sum_{k=i}^{j} p_k \right).$$

Для ускорения вычисления суммы частот используется массив префиксных сумм.

```
static int OptimalBst(int[] keys, int[] freq)
{
    int n = keys.Length;
    int[,] cost = new int[n, n];
    int[] prefix = new int[n + 1];

    for (int i = 0; i < n; i++)
        prefix[i + 1] = prefix[i] + freq[i];

    for (int length = 1; length <= n; length++)
    for (int left = 0; left + length <= n; left++)
    {
        int right = left + length - 1;
        cost[left, right] = int.MaxValue;
        int sum = prefix[right + 1] - prefix[left];

        for (int root = left; root <= right; root++)
        {
            int leftCost = root > left ? cost[left, root - 1] : 0;
            int rightCost = root < right ? cost[root + 1, right] : 0;
            int current = leftCost + rightCost + sum;
            cost[left, right] = Math.Min(cost[left, right], current);
        }
    }

```

```
    return cost[0, n - 1];
}
```

Листинг 2: Построение оптимального дерева поиска

2.4 Результат

Оптимальным корнем является ключ 30. Слева от него располагается поддерево с корнем 10 и правым сыном 20, справа --- ключ 40.

Стоимость дерева:

$$C = 4 \cdot 1 + 3 \cdot 2 + 2 \cdot 3 + 1 \cdot 3 = 18.$$

Минимальная суммарная стоимость поиска равна 18.

Временная сложность алгоритма составляет $O(n^3)$, пространственная сложность --- $O(n^2)$.

3 Задание 3. Раскрой бумажного бруска

3.1 Условие

Дан бумажный брусок длины L и N заказов. Заказ характеризуется длиной требуемого куска d_i и ценой p_i. Каждый выполненный заказ требует одного реза стоимостью C. Необходимо выбрать заказы с максимальной прибылью:

$$P = \sum p_i - C \cdot k,$$

где k --- число выполненных заказов.

3.2 Исходные данные

В качестве теста использованы $L = 20$ см и $C = 2$ рубля.

Таблица 3: Заказы на куски бруска

№	Длина, см	Цена, руб.
1	6	11
2	8	14
3	5	9
4	7	13
5	4	6

3.3 Алгоритм решения

Для каждого заказа рассчитывается эффективная прибыль $p_i - C$. Затем задача решается как задача о рюкзаке по одному ограничению: $dp[x]$ хранит максимальную прибыль при занятой длине x. Перебор длины выполняется справа налево, поэтому каждый заказ можно использовать не более одного раза.

```
static (int profit, List<int> orders) Cutting(
    int length, (int d, int p)[] orders, int cutCost)
{
    int[] dp = new int[length + 1];
    List<int>[] chosen = new List<int>[length + 1];

    for (int i = 0; i <= length; i++)
        chosen[i] = new List<int>();

    for (int i = 0; i < orders.Length; i++)
    {
        int size = orders[i].d;
        int gain = orders[i].p - cutCost;

        for (int used = length; used >= size; used--)
        {
            int candidate = dp[used - size] + gain;

            if (candidate > dp[used])
            {
                dp[used] = candidate;
                chosen[used] = new List<int>(chosen[used - size]);
                chosen[used].Add(i + 1);
            }
        }
    }

```

```
    return (dp[length], chosen[length]);
}
```

Листинг 3: Динамическое программирование для раскроя

3.4 Результат

Алгоритм выбирает заказы 2, 3 и 4:

$$d = 8 + 5 + 7 = 20 \text{ см},$$

$$\sum p_i = 14 + 9 + 13 = 36 \text{ руб.}, \qquad C \cdot k = 2 \cdot 3 = 6 \text{ руб.}$$

Итоговая максимальная прибыль:

$$P = 36 - 6 = 30 \text{ руб.}$$

Временная сложность алгоритма равна $O(NL)$, пространственная сложность --- $O(L)$.

4 Сравнение результатов

Таблица 4: Сравнение реализованных алгоритмов

Задача	Состояние динамики	Время	Память	Результат
Эвакуация	предмет, вес, время	$O(nWT)$	$O(nWT)$	19 \$
Дерево поиска	отрезок ключей	$O(n^3)$	$O(n^2)$	стоимость 18
Раскрой	занятая длина	$O(NL)$	$O(L)$	30 руб.

Во всех задачах решение строится из оптимальных решений меньших подзадач. В задаче эвакуации используются два ограниченных ресурса, в задаче построения дерева состояние задаётся границами отрезка ключей, а в задаче раскроя --- текущей занятой длиной материала.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы изучен метод динамического программирования и реализованы три алгоритма. Для задачи эвакуации найден набор предметов максимальной ценности при ограничениях по весу и времени. Для заданных частот построено оптимальное бинарное дерево поиска с минимальной стоимостью обращений. Для задачи раскроя выбраны заказы, обеспечивающие максимальную прибыль с учетом стоимости резов.

Полученные результаты показывают, что динамическое программирование позволяет эффективно решать задачи, в которых оптимальный ответ может быть сформирован из оптимальных ответов меньших подзадач.